EvidenceFit

AI-Powered Evidence-Based Fitness Platform

Technical Project Report

Repository	github.com/hrithik-dev8/GenAI_Project
App URL	https://gen-ai-project-neon.vercel.app/
Frontend	React 18 + Vite + TailwindCSS
Backend	Supabase Edge Functions (Deno)
AI Models	Groq Llama 3.3 70B + OpenAI Embeddings
Deployment	Vercel + Supabase
Architecture	RAG + Multi-Agent Pipeline

Hrithik Dev	M01096886
Sourav Chindarmony	M01094141
Netra Uchil	M01089309

Log In Credentials	**(Optional, can create own account)**
Email	devhrithik8@gmail.com
Password	Ivan@2026

1. Project Overview

EvidenceFit is a full-stack, AI-powered fitness coaching web application that generates personalized, peer-reviewed, evidence-backed workout and meal plans. Unlike generic fitness apps, every recommendation EvidenceFit makes is grounded in actual sports science research, retrieved at query time via Retrieval-Augmented Generation (RAG).

The platform bridges the gap between academic exercise science and everyday users by surfacing relevant research abstracts inline alongside every AI-generated plan, providing transparency and credibility that generic fitness apps cannot match.

> *Core Mission: Deliver personalised, evidence-based fitness and nutrition coaching at scale; making sports science accessible to everyone, not just elite athletes with access to professional coaches.*

1.1 Key Differentiators

- Research-grounded output: 30 curated sports science abstracts embedded in a vector store; every AI response cites sources with inline [n] references
- Multi-agent specialization: separate AI agents handle vision, workout programming, sports nutrition, and coaching rationale
- Model-agnostic architecture: switching LLM providers requires only updating environment variables
- Safety-first design: prompt injection guardrails, emergency detection, and domain scoping built into every Edge Function
- Serverless, scalable deployment: zero-ops infrastructure via Vercel and Supabase

1.2 Target Users

- Fitness enthusiasts seeking evidence-backed personalised plans
- Coaches and trainers who want AI-assisted plan generation grounded in research
- Researchers exploring applied AI in sports science

2. System Architecture

EvidenceFit follows a modern serverless architecture that separates concerns cleanly across the frontend, backend, AI pipeline, and data layers. The system is designed for high availability, easy scalability, and rapid iteration.

2.1 Architecture Overview

Layer	Technology	Responsibility
Presentation	React 18 + Vite + TailwindCSS	User interface, onboarding flow, chat, plan display
Hosting	Vercel (auto-deploy via GitHub)	Static site delivery, CDN, CI/CD
Auth & Storage	Supabase Auth + PostgreSQL	User sessions, profiles, plans, research corpus
Vector Store	pgvector (PostgreSQL extension)	High-performance cosine similarity search on embeddings
Embeddings	OpenAI text-embedding-3-small	Encoding research abstracts and user queries
AI Inference	Groq API — Llama 3.3 70B	Plan synthesis, chat, streaming responses
Backend Logic	Supabase Edge Functions (Deno)	RAG retrieval, multi-agent orchestration, guardrails

2.2 Data Flow

The request lifecycle follows this sequence:

- User submits a query or plan request via the React frontend
- Request is routed to the appropriate Supabase Edge Function over HTTPS
- Edge Function encodes the query using OpenAI text-embedding-3-small
- pgvector performs cosine similarity search against the 30-abstract research corpus
- Top-5 relevant research abstracts are retrieved and injected into the LLM prompt
- Groq (Llama 3.3 70B) generates a response with inline citations streamed back to the client
- Frontend renders the streamed response progressively in real time

> *Architecture Principle: All AI logic lives in Edge Functions; the frontend is intentionally kept thin, containing only rendering and UX logic. This keeps the AI pipeline portable and independently deployable.*

3. Key Components

3.1 Frontend — React + Vite + TailwindCSS

The frontend is a single-page application hosted on Vercel with automatic deployments triggered by every push to the main branch on GitHub.

Technology Stack

Package	Purpose
React 18	Core UI framework with concurrent rendering
Vite	Lightning-fast build tool and dev server
TailwindCSS	Utility-first styling for rapid, consistent UI
React Router	Client-side routing between views
TanStack Query	Server state management, caching, and sync
Radix UI	Accessible, unstyled component primitives

Key Features

- User onboarding: collects fitness goals, activity level, dietary preferences, and gym equipment
- AI Chat: real-time streaming chat interface with inline citation rendering
- Plan Display: structured rendering of workout schedules and macro-based meal plans
- Meal Tracking: logging and history of nutrition intake
- Responsive Design: mobile-first layout with TailwindCSS

3.2 Database — Supabase / PostgreSQL

The database layer uses Supabase's managed PostgreSQL instance, extended with pgvector for efficient vector similarity search. Row-Level Security (RLS) policies ensure each user can only access their own data.

Schema Highlights

Table	Contents
users / profiles	Auth identity, fitness goals, preferences, physical stats
research_corpus	30 sports science abstracts with title, text, and vector column
workout_plans	AI-generated resistance training programs linked to a user

Table	Contents
meal_plans	Calorie and macro targets with daily meal structures
chat_history	Persisted conversation threads for continuity across sessions

The pgvector extension enables the IVFFlat index for approximate nearest-neighbour search over 1536-dimensional embedding vectors, making retrieval sub-millisecond at this corpus scale.

3.3 RAG Vector Store

The research corpus consists of 30 carefully curated sports science abstracts covering resistance training, hypertrophy, HIIT, nutrition timing, protein synthesis, recovery, and more. Each abstract is pre-embedded using OpenAI text-embedding-3-small and stored in the pgvector column.

> *RAG Design Choice: Using a curated, domain-specific corpus (rather than a large general corpus) ensures that retrieved context is always directly relevant to fitness — reducing hallucination risk and improving citation quality.*

At inference time, the user's query or profile data is embedded and the top-5 most semantically similar abstracts are retrieved via cosine similarity. These are concatenated into the LLM system prompt with a strict instruction to cite sources inline.

3.4 AI Chat Edge Function

The chat Edge Function handles free-form user questions about fitness, nutrition, and health. It includes several safety and quality layers:

Safety Controls

- Prompt Injection Guardrails: regex patterns detect attempts to override system instructions or exfiltrate context
- Emergency Detection: keywords related to medical emergencies trigger a redirect to professional help rather than an AI response
- Domain Scoping: the system prompt restricts the model to fitness and nutrition topics, refusing off-topic requests

Response Quality

- RAG-augmented context: top-5 relevant research abstracts injected before every response

- Mandatory inline citations: [n] reference format enforced in the system prompt
- Streaming output: response is streamed token-by-token to the frontend for a real-time feel

3.5 Multi-Agent Plan Generator

Plan generation is the most complex pipeline in EvidenceFit. It uses a sequential multi-agent architecture where each agent specialises in a distinct domain, feeding its output to the next agent.

Agent	Role	Responsibilities
Agent 0	Vision Analyst	Analyses gym photos to inventory available equipment; outputs a structured equipment list used by Agent 1
Agent 1	Workout Programmer	Designs a periodised resistance training program based on user goals, training history, and equipment; cites relevant research
Agent 2	Sports Nutritionist	Calculates Total Daily Energy Expenditure (TDEE), sets calorie and macro targets, structures daily meal timing
Agent 3	Coach Synthesizer	Writes a narrative coaching rationale tying together the workout and nutrition plan, with inline citations to research

Agents communicate by passing structured JSON between Edge Function invocations. Each agent receives the full user profile, the outputs of all preceding agents, and the top-5 RAG-retrieved research abstracts relevant to its domain.

4. Configuration & Model Switching

EvidenceFit is intentionally decoupled from any specific AI provider. All model configuration is managed through Supabase secrets (environment variables), making it trivial to swap providers without touching application code.

Environment Variable	Purpose
GROQ_API_KEY	API key for Groq inference (Llama 3.3 70B)
OPENAI_API_KEY	API key for text-embedding-3-small
GROQ_BASE_URL	Base URL — swap to switch to OpenAI or Anthropic
MODEL_ID	Model name string — change to target a different model

Environment Variable	Purpose
SUPABASE_URL	Supabase project URL for database and auth
SUPABASE_ANON_KEY	Public anon key for client-side Supabase access

> *To migrate from Groq to OpenAI or Anthropic: update GROQ_BASE_URL to the target provider's endpoint, update GROQ_API_KEY with the new provider's key, and update MODEL_ID to the desired model name. No code changes required.*

5. Deployment

EvidenceFit uses a fully managed, zero-ops deployment stack. There are no servers to provision or patch — all infrastructure is handled by Vercel and Supabase.

5.1 Frontend — Vercel

- Auto-deploy triggered on every push to the main branch via GitHub integration
- Preview deployments created for every pull request, enabling safe feature testing
- Global CDN delivery with automatic SSL
- Environment variables managed in the Vercel dashboard

5.2 Backend — Supabase Edge Functions

- Edge Functions deployed via the Supabase CLI: supabase functions deploy <name>
- Deno runtime — secure, TypeScript-native, no npm dependency management required
- Auto-scaling — Supabase manages concurrency and cold start mitigation
- Secrets managed via Supabase Vault — never exposed to client code

5.3 Database — Supabase Managed PostgreSQL

- Fully managed Postgres with automated backups and point-in-time recovery
- pgvector extension enabled at the project level
- Row-Level Security policies enforced at the database layer
- Real-time subscriptions available for live plan update notifications

5.4 CI/CD Flow

Developer pushes to GitHub → Vercel auto-builds and deploys the frontend → Supabase CLI deploys updated Edge Functions → Database migrations applied via supabase db push. The entire pipeline from commit to production takes under 2 minutes.

6. Safety & Quality Controls

Safety and output quality are first-class concerns in EvidenceFit. The system implements multiple complementary layers of protection and validation.

6.1 Input Safety

Control	Implementation
Prompt Injection Prevention	Regex patterns detect and block attempts to override system instructions or leak context
Emergency Detection	Keywords (e.g. 'chest pain', 'can't breathe') trigger a safety redirect rather than AI response
Domain Scoping	System prompt instructs the model to refuse off-topic requests (e.g. coding, politics, finance)
Input Sanitization	User inputs are sanitized before being interpolated into prompts

6.2 Output Quality

Control	Implementation
Mandatory Citations	System prompt mandates [n] inline citation format; model cannot respond without citing sources
RAG Grounding	Every response is grounded in retrieved research abstracts; reducing hallucination risk
Agent Specialization	Separate agents for workout vs. nutrition prevents cross-domain confusion and scope creep
Structured Output	Plan agents are instructed to return structured JSON for deterministic rendering

> *Quality Philosophy: Rather than relying solely on model capability, EvidenceFit constrains the solution space at the prompt level, making it structurally difficult for the model to produce uncited, hallucinated, or off-topic responses.*

7. Full Technology Stack

Category	Technology	Notes
Frontend Framework	React 18	Concurrent features, Suspense
Build Tool	Vite	ESM-native, HMR
Styling	TailwindCSS	Utility-first, JIT compiler
Routing	React Router v6	File-based routing patterns
Data Fetching	TanStack Query	Caching, background sync
UI Components	Radix UI	Accessible, unstyled primitives
Frontend Hosting	Vercel	Auto-deploy, CDN, preview envs
Backend Runtime	Deno (Supabase Edge)	TypeScript, secure, no node_modules
Database	PostgreSQL (Supabase)	Managed, RLS, real-time
Vector Search	pgvector	IVFFlat index, cosine similarity
Auth	Supabase Auth	JWTs, RLS integration
Embeddings	OpenAI text-embedding-3-small	1536-dim vectors
LLM Inference	Groq — Llama 3.3 70B	Fast inference, streaming
Secret Management	Supabase Vault	Encrypted env variables
Version Control	GitHub	Source of truth, CI/CD trigger